

Name:

Nirbhay

Subject:

Machine Learning

Assignment:

Linear Regression and Gradient Descent

Task 1: Simple Linear Regression from Scratch in C

Objective

To implement Simple Linear Regression from scratch using the C programming language without using any machine learning libraries.

Dataset

A self-created dataset consisting of 20 samples was used.

Input Feature: X Value

Target Variable: Y Value

The dataset follows the relationship:

$$y = 2x + 5$$

Methodology

The following components were implemented manually:

- Weight Initialization
- Bias Initialization
- Prediction Function

- Mean Squared Error (MSE) Loss Function
- Gradient Calculation
- Gradient Descent Update Rule
- Training Loop

Results:

Initial Parameters

- Weight = 0
- Bias = 0
- Loss $\approx$ 671

Final Parameters

- Weight = 2.00
- Bias = 4.99
- Loss = MSE $\approx$ 0

Learned Equation

$$\hat{y} = 2.00x + 5.00$$

The learned equation closely matches the actual relationship present in the dataset.

```

=====
DATASET: Study Hours vs Marks Scored
Number of samples: 20

Initial Parameters:
Initial Weight (w): 0.000000
Initial Bias (b) : 0.000000
Initial Loss (MSE): 671.000000

Training in Progress...

Epoch      Weight (w)      Bias (b)      Loss (MSE)
-----
Epoch 0    | 0.154000 | 0.034000 | 517.067500
Epoch 1000 | 1.913156 | 3.954096 | 0.442893
Epoch 2000 | 1.984299 | 4.808805 | 0.007308
Epoch 3000 | 1.997173 | 4.963621 | 0.000121
Epoch 4000 | 1.999502 | 4.991640 | 0.000002
Epoch 5000 | 1.999924 | 4.996706 | 0.000000

Final Parameters (After Training):
Final Weight (w): 1.999924
Final Bias (b) : 4.996706
Final Loss (MSE): 0.000000
+-----+

```

Discussion

1. How did the weight change during training?

Initially, the weight was set to **0**. During gradient descent, the weight gradually increased and moved toward the optimal value (approximately **2**). This happened because the algorithm continuously adjusted the weight to reduce prediction errors.

2. How did the bias change during training?

The bias started from **0** and gradually increased toward the optimal value (approximately **5**). The bias adjustment helped shift the regression line upward so that predictions better matched the actual data.

3. Why did the loss decrease over time?

The loss decreased because gradient descent updates the weight and bias in the direction that minimizes the Mean Squared Error (MSE). With every iteration, the model predictions became closer to the actual values, reducing the overall error .

4. Does the final equation represent the dataset trend?

Yes. The learned equation accurately represents the trend present in the dataset.

Task 2: Gradient Descent and Learning Rate Exploration

Objective

To study the impact of different learning rates on the convergence behavior of Gradient Descent.

Function Used

$$f(x) = x^2 + x$$

Learning Rates Tested

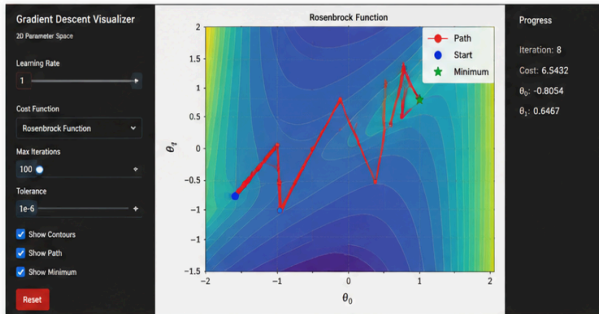
1. $\alpha = 1.0$
2. $\alpha = 0.1$
3. $\alpha = 0.01$
4. $\alpha = 0.001$

Observations

Learning Rate	Speed of Convergence	Stability	Overshooting	Final Position
1.0	Very Fast	Unstable	High	Often diverges
0.1	Fast	Good	Minor	Reaches optimum quickly
0.01	Moderate	Very Stable	None	Reaches optimum accurately
0.001	Slow	Extremely Stable	None	May require many iterations

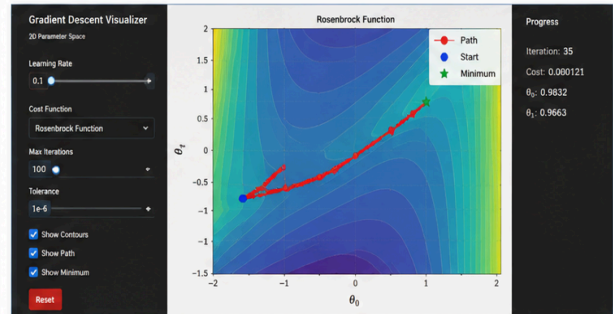
Task 2: Gradient Descent and Learning Rate Exploration

1. Learning Rate = 1.0



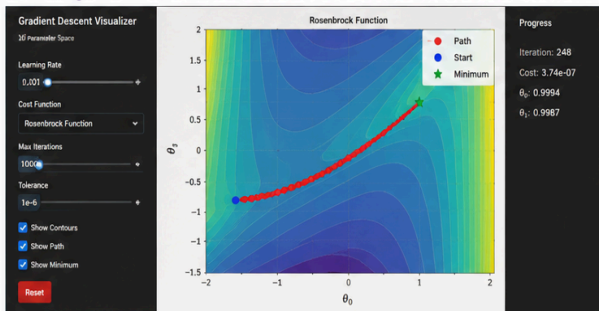
- **Speed of Convergence:** Very Fast (but unstable)
- **Stability:** Poor
- **Overshooting:** High (large jumps, oscillates a lot)
- **Final Position:** Does not converge to minimum
- **Path Shape:** Zig-Zag, highly erratic

2. Learning Rate = 0.1



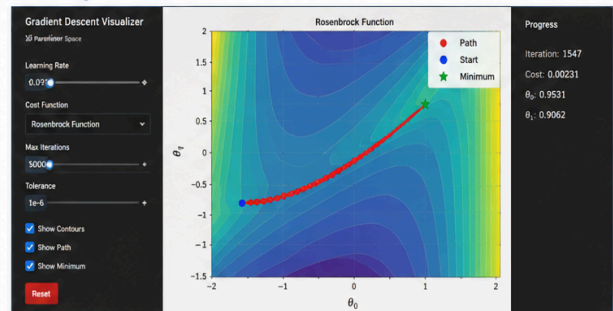
- **Speed of Convergence:** Fast
- **Stability:** Good
- **Overshooting:** Small (very slight oscillations)
- **Final Position:** Converges near the minimum
- **Path Shape:** Smooth, slightly curved

3. Learning Rate = 0.01



- **Speed of Convergence:** Moderate
- **Stability:** Very Stable
- **Overshooting:** None
- **Final Position:** Converges very close to minimum
- **Path Shape:** Smooth and consistent

4. Learning Rate = 0.001



- **Speed of Convergence:** Slow
- **Stability:** Extremely Stable
- **Overshooting:** None
- **Final Position:** Converges slowly toward minimum
- **Path Shape:** Very slow and smooth

Discussion

1. Which learning rate converged the fastest?

Learning rate 0.1 converged the fastest while maintaining stability.

2. Which learning rate was the most stable? Learning rate 0.01 showed the most stable convergence behavior.

3. What happens when the learning rate is too large?

A very large learning rate causes overshooting and instability, preventing proper convergence.

4. What happens when the learning rate is too small?

A very small learning rate results in extremely slow convergence and requires many iterations.

5. Why is Gradient Descent considered an optimization algorithm?

Gradient Descent minimizes the loss function by iteratively adjusting model parameters toward the optimal solution.

Task 3: Multiple Linear Regression from Scratch Using NumPy

Objective

To implement Multiple Linear Regression using NumPy and compare the results with Scikit-Learn.

Dataset

Insurance Dataset

Features Used

- Age
- BMI
- Children

Target Variable

- Charges
- Total samples: 1338

Methodology

The following components were implemented manually using NumPy:

- Weight Vector Initialization
- Bias Initialization
- Prediction Function
- Mean Squared Error (MSE)
- Gradient Calculation
- Gradient Descent
- Model Training

Feature scaling was applied before training.

Results:

Custom NumPy Model

Mean Squared Error (MSE): 131201335.66

Scikit-Learn Model

Mean Absolute Error (MAE): 9181.31

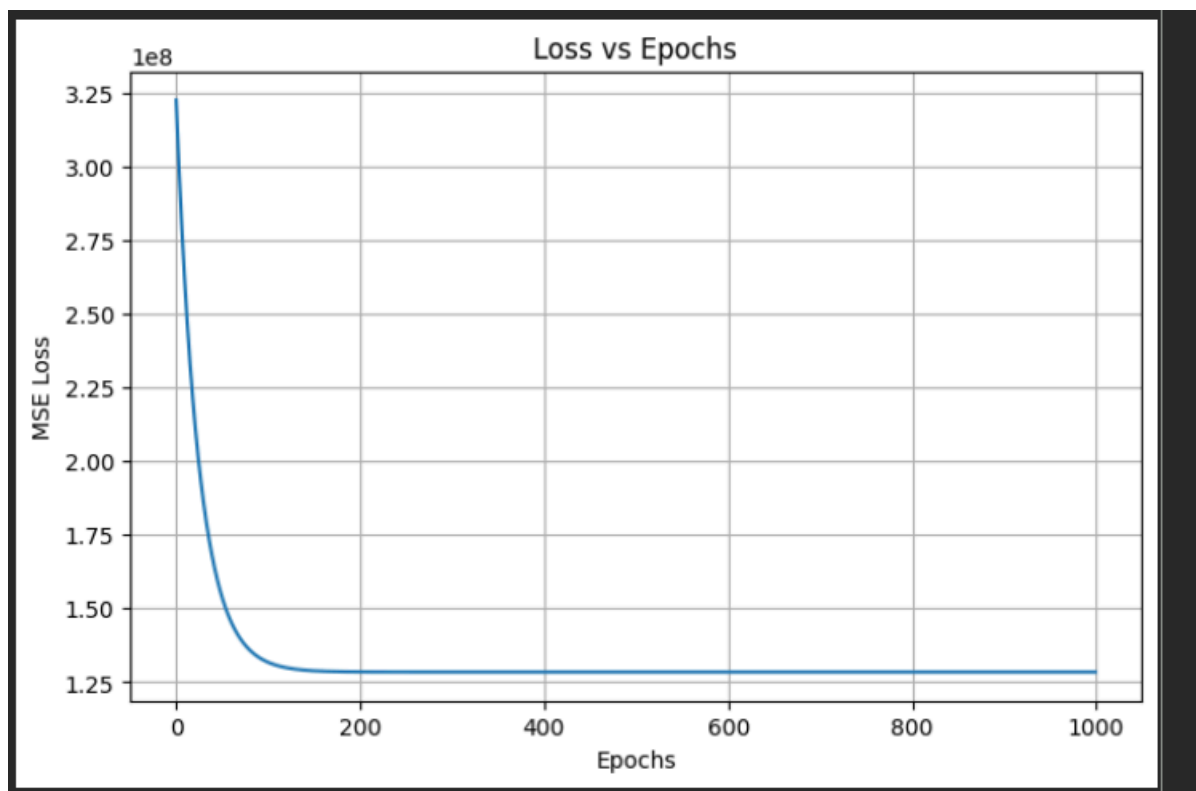
Mean Squared Error (MSE): 131201335.65

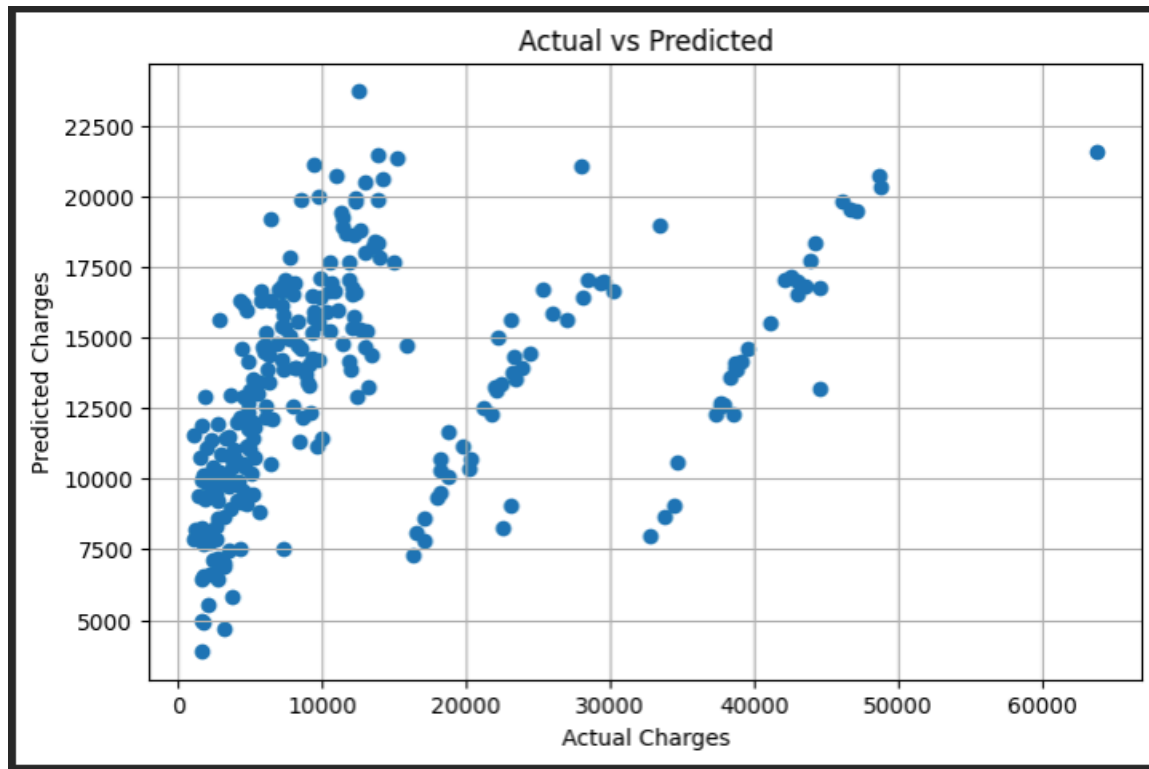
Coefficients: [684.30]

Graphs

The following graphs were generated:

1. Loss vs Epochs
2. Actual vs Predicted Values





Discussion

1. How similar were the results?

The custom NumPy model and Scikit-Learn model produced nearly identical MSE values, indicating correct implementation.

2. Why might there be small differences?

Small differences may arise due to numerical precision and implementation details.

3. What challenges were faced?

The major challenges included implementing gradient descent correctly, handling multiple features, and selecting an appropriate learning rate.

4. Why are learning rate and feature scaling important?

Feature scaling improves convergence speed, while the learning rate controls the size of parameter updates during training.
